Week 1: Basics – Variables, Data Types, Input/Output, Loops, Conditionals

Q1. Write a python program to swap two numbers

In [21]:
```
a = 10
b = 15
print("Before swap",a,b)
a , b = b , a
print("After swap",a,b)
```

```
Before swap 10 15
After swap 15 10
```

Q2. Take user input and display it to the user

In [9]:
```
data = input("enter something")
print(data)
```

```
enter
```

Q3. Write a program to check if a number is even or odd.

In [12]:
```
num = int(input("enter a number"))
if num % 2 == 0:
    print(f"{num} is a even number")
else:
    print(f"{num} is a odd number")
```

```
2 is a even number
```

Q4. Create a program that prints the multiplication table of a given number.

In [13]:
```
num = int(input("enter the number you want to find table of"))
for i in range(1,11):
    print(f"{num} * {i} = {num* i}")
```

```
10 * 1 = 10
10 * 2 = 20
10 * 3 = 30
10 * 4 = 40
10 * 5 = 50
10 * 6 = 60
10 * 7 = 70
10 * 8 = 80
10 * 9 = 90
10 * 10 = 100
```

Q5. Write a program to find the largest of three numbers.

In [17]:
```
x = 10
y = 13
x = 15
if x > y and x > z:
    print(f"{x} is the largest number")
elif y>x and y>z:
    print(f"{y} is the largest number")
elif z>x and z>y:
    print(f"{z} is the largest number")
else:
    print("INVALID VALUES FOR COMPARISION")
```

```
15 is the largest number
```

Q6. Convert temperature from Celsius to Fahrenheit.

In [18]:
```
temp = int(input("Enter the temperatur in Celsius"))
Fahrenheit = temp * 1.8
print(Fahrenheit)
```

```
18.0
```

Q7. Write a program to calculate the factorial of a number using a loop.

In [19]:
```
num = int(input("Enter the number you want to find factorial of"))
fac = 1
for i in range (1,num+1):
    fac = fac * i
print(fac)
```

```
120
```

Q8. Create a program to count the number of vowels in a string.

In [3]:
```
string = " There are 8 planets in the solar system"
print(string.count("a"))
print(string.count("e"))
print(string.count("i"))
print(string.count("o"))
print(string.count("u"))
```

```
3
6
1
1
0
```

In [6]:
```
string = " There are 8 planets in the solar system"
vowel = "aeiou"
count = 0
# using for loop to access all the characters of the string and comparing it w
for character in string:
    # now comparing each charcters with vowels if a character is present in th
    if character in vowel:
        count += 1
print(f"The number of vowel in the sting is: {count}")
```

```
The number of vowel in the sting is: 11
```

Q9. Write a Python script to reverse a given string.

In [24]:
```
a = input("Enter the string that you want to reverse")
print("Before Reverse :",a)
print("After Reverse :",a[::-1])
```

```
Before Reverse : swopnim
After Reverse : minpows
```

Q10. Check if a number is a palindrome.

In [2]:
```
a = input("Enter the number that you want to check whether it's palindrome or
b = a[::-1]
if a==b :
    print("palindrome")
else :
    print("not palindrome")
```

```
palindrome
```

Q11. Write a program to find the sum of first N natural numbers.

In [26]:
```
n = int(input("Enter the value for n :"))
s = 0
for i in range(1,n+1):
    s+=i
print(s)
```

```
15
```

Q12. Create a number guessing game.

In [1]:
```
import random
x = random.randint(1,10)
while True:
    try:
        y = int(input("Enter a number between one to ten"))
        if x > y :
```

```
            print(f"{y} is less than the acutal number")
        elif y > x :
            print(f"{y} is more than the actual number")
        else:
            print("CONGRATULATIONS YOUR GUESS WAS CORRECT")
            break
    except ValueError:
        # Handle cases where the user enters non-integer input
        print("That's not a valid number. Please enter a whole number.")
```

```
10 is more than the actual number
5 is more than the actual number
4 is more than the actual number
3 is more than the actual number
2 is more than the actual number
CONGRATULATIONS YOUR GUESS WAS CORRECT
```

Q13. Write a program to print all prime numbers between 1 and 100.

In [12]:
```
for i in range(1,101):
    count = 0
    for j in range(1, i+1):
        if i % j == 0 :
            count += 1
    if count == 2:
         print(f"{i} is a prime number")
```

```
2 is a prime number
3 is a prime number
5 is a prime number
7 is a prime number
11 is a prime number
13 is a prime number
17 is a prime number
19 is a prime number
23 is a prime number
29 is a prime number
31 is a prime number
37 is a prime number
41 is a prime number
43 is a prime number
47 is a prime number
53 is a prime number
59 is a prime number
61 is a prime number
67 is a prime number
71 is a prime number
73 is a prime number
79 is a prime number
83 is a prime number
89 is a prime number
97 is a prime number
```

Q14. Check if a given year is a leap year or not.

- First, check if the year is divisible by 400. If it is, it's a leap year.
- If not, check if it's divisible by 100. If it is, it's not a leap year.
- If not, check if it's divisible by 4. If it is, it's a leap year.
- If none of these conditions are met, it is not a leap year.

In [19]:
```
year = int(input("Enter the year you want to check if it's a leap year or not:

if (year % 400 == 0) or (year % 100 != 0 and year % 4 == 0):
    print(f"{year} is a leap year")
else :
    print(f"{year} is not a leap year")
```

```
2000 is a leap year
```

Q15. Create a program to print the Fibonacci series up to N terms.

In [3]:
```
def fibonacci_series(n):
    # handel edge cases for n = 0 and n = 1
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    # Initialize the first two numbers
    series = [0,1]

    # Generate the series up to n terms
    while len(series) < n:
        next_number = series[-1] + series[-2]
        series.append(next_number)

    return series

# Example usage:
n_terms = 10
fib_sequence = fibonacci_series(n_terms)
print(f"Fibonacci series up to {n_terms} terms: {fib_sequence}")
```

```
Fibonacci series up to 10 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Q16. Write a program to find the GCD of two numbers.

In [36]:
```
def find_gcd(a, b):
```

```
    """
    Finds the greatest common divisor (GCD) of two numbers using the Euclidean
    The Euclidean algorithm is based on the principle that the GCD of two numb
    does not change if the larger number is replaced by its difference with th
    smaller number. This is repeated until one of the numbers is zero.

    Args:
        a (int): The first number.
        b (int): The second number.

    Returns:
        int: The greatest common divisor of a and b.
    """
    # The loop continues as long as b is not 0.
    # The Euclidean algorithm relies on the property: GCD(a, b) = GCD(b, a % b
    while b!= 0:
        # The line below is a concise way to swap a and b,
        # and update b with the remainder of a divided by b.
        # It's equivalent to:
        # temp = b
        # b = a % b
        # a = temp
        a, b = b, a % b

    # When the loop terminates, a holds the GCD.
    return a

# Example usage:
num1 = 54
num2 = 24

# Call the function and store the result
result = find_gcd(num1, num2)

# Print the result in a user-friendly format
print(f"The GCD of {num1} and {num2} is: {result}")

# Another example:
num3 = 48
num4 = 180
print(f"The GCD of {num3} and {num4} is: {find_gcd(num3, num4)}")
```

```
The GCD of 54 and 24 is: 6
The GCD of 48 and 180 is: 12
```

Q17. Write a program to find the lcm

In [39]:

```
import math

def find_gcd(a, b):
    """
    Finds the greatest common divisor (GCD) of two numbers using the
    Euclidean algorithm. This function is a prerequisite for finding the LCM.
```

```
    """
    while b:
        a, b = b, a % b
    return a

def find_lcm(a, b):
    """
    Finds the least common multiple (LCM) of two numbers.

    The formula used is:
    LCM(a, b) = (|a * b|) / GCD(a, b)

    This formula works because the product of two numbers is equal to the
    product of their GCD and LCM.

    Args:
        a (int): The first number.
        b (int): The second number.

    Returns:
        int: The least common multiple of a and b.
    """
    # If either number is zero, their LCM is 0.
    if a == 0 or b == 0:
        return 0

    # Use the formula to calculate LCM.
    # We use abs() to handle potential negative inputs, and // for integer div
    gcd = find_gcd(a, b)
    lcm = abs(a * b) // gcd
    return lcm

# Example usage:
num1 = 12
num2 = 18

# Call the function and store the result
result_lcm = find_lcm(num1, num2)

# Print the result in a user-friendly format
print(f"The LCM of {num1} and {num2} is: {result_lcm}")

# Another example:
num3 = 15
num4 = 20
print(f"The LCM of {num3} and {num4} is: {find_lcm(num3, num4)}")

# Example with a larger number set
num5 = 48
num6 = 180
print(f"The LCM of {num5} and {num6} is: {find_lcm(num5, num6)}")
```

```
The LCM of 12 and 18 is: 36
The LCM of 15 and 20 is: 60
The LCM of 48 and 180 is: 720
```

Q18. Check whether a character is a vowel or consonant.

In [49]:
```python
# Some what similar to question number
string = "A quick brown fox jumps over the lazy dog"
vowel= "aeiou"
string_clean = string.lower().replace(" ","")
for character in string_clean:
    if character in vowel:
        print(f"{character} in string is a vowel")
    else:
        print(f"{character} in string is a consonant")
```

```
a in string is a vowel
q in string is a consonant
u in string is a vowel
i in string is a vowel
c in string is a consonant
k in string is a consonant
b in string is a consonant
r in string is a consonant
o in string is a vowel
w in string is a consonant
n in string is a consonant
f in string is a consonant
o in string is a vowel
x in string is a consonant
j in string is a consonant
u in string is a vowel
m in string is a consonant
p in string is a consonant
s in string is a consonant
o in string is a vowel
v in string is a consonant
e in string is a vowel
r in string is a consonant
t in string is a consonant
h in string is a consonant
e in string is a vowel
l in string is a consonant
a in string is a vowel
z in string is a consonant
y in string is a consonant
d in string is a consonant
o in string is a vowel
g in string is a consonant
```

Q19. Write a program to calculate the sum of digits of a number.

In [54]:
```
num = input("Enter a number")
sum = 0
for digits in num:
    sum = sum + int(digits)
print(sum)
```

```
6
```

Q20. Create a program to find the second largest number in a list.

In [121…
```
a = [1,2,3,49,5,76,33,44]
a = sorted(a)
print(a)
length = len(a)
print("the length of the list is",length)
# the second largest number in the list will be the second last item in the li
# the second last item will be in the length - 2 index
second_largest_number = a[length-2]
print("is the second largest number in the list",second_largest_number)
```

```
[1, 2, 3, 5, 33, 44, 49, 76]
the length of the list is 8
is the second largest number in the list 49
```

Q21. Write a program to count the number of digits in an integer.

In [65]:
```
number = input("enter the number which you want to count digit of")
count = 0
for digits in number:
    count += 1
print(f"The numbers of digits in the number is : {count}")
```

```
The numbers of digits in the number is : 3
```

Q22. Create a program to print all Armstrong numbers between 1 to 1000.

- Take a number.
- Count how many digits it has → that becomes the power.
- Raise each digit of the number to that power.

- Add them all up.
- If the result equals the original number → it's an Armstrong number.

In [77]:
```
number = input("Armstrong number checker, Enter a number")
power = len(number)
print(power)
total = 0
for digit in number:
    total = total + int(digit)**power
if total == int(number):
    print(f"{number} is a armstrong number")
else:
    print(f"{number} is not a armstrong number")
```

```
3
153 is a armstrong number
```

Q23. Write a Python program to print a pattern of stars in a triangle.

In [98]:
```
row_in_star = int(input("Enter the no of rows in the star you want"))

for i in range( 1 , row_in_star+1 ):
    print("*" * i )
```

```
*
**
***
****
*****
******
*******
********
*********
**********
```

In [97]:
```
##### row_in_star = int(input("Enter the number of rows in the star you want")
for i in range (1 , row_in_star + 1 ):
    x = row_in_star + 1 - i
    print("*" * x)
```

```
**********
*********
********
*******
******
*****
****
***
**
*
```

In [96]:
```
rows = int(input("enter the numbers of rows you want in the pyramid"))

for i in range(rows):
    space = " " * (rows -  i - 1)
    symbol = "*" * (2 * i + 1)
    print(space + symbol)
```

```
        *
       ***
      *****
     *******
    *********
   ***********
  *************
 ***************
*****************
*******************
```

In [99]:
```
rows = int(input("enter the numbers of rows you want in the pyramid"))

for i in range(rows):
    space = " " * (rows -  i - 1)
    if i % 2 == 0 :
        symbol = "#" * (2 * i + 1)
    else:
        symbol = "*" * (2 * i + 1)
    print(space + symbol)
```

```
        #
       ***
      #####
     *******
    #########
   ***********
  #############
 ***************
#################
*******************
```

In [107…
```
rows = int(input("Enter the number of rows you want in the pyramid: "))

for i in range(rows):
    # spaces for alignment
    space = " " * (rows - i - 1)

    # build the symbols for this row
    symbols = ""
    for j in range(2 * i + 1):  # total symbols in row
        if j % 2 == 0:
            symbols += "#"
        else:
            symbols += "*"

    # print the row
```

```
    print(space + symbols)
```

```
            #
           #*#
          #*#*#
         #*#*#*#
        #*#*#*#*#
       #*#*#*#*#*#
      #*#*#*#*#*#*#
     #*#*#*#*#*#*#*#
    #*#*#*#*#*#*#*#*#
   #*#*#*#*#*#*#*#*#*#
  #*#*#*#*#*#*#*#*#*#*#
 #*#*#*#*#*#*#*#*#*#*#*#
#*#*#*#*#*#*#*#*#*#*#*#*#
```

Q24. Create a calculator app using if-else.

In [118…
```
a = 3
b = 4
task = input("Enter the task: ").lower()

if task == 'add' or task == 'sum':
    print(a + b)
elif task == 'sub' or task == 'difference':
    print(a - b)
elif task == 'multiply' or task == 'product':
    print(a * b)
elif task == 'division' or task == 'div':
    print(a / b)
else:
    print("ERROR")
```

```
-1
```

Q25. Write a program to display the ASCII value of a character.

In [120…
```
character = input("Enter a single character")
ascii_value = ord(character)
print(f"The ascii value of `{character}` is {ascii_value}")
```

```
The ascii value of `1` is 49
```

In [125…
```
# program to find the ascii value of all the character of the string
string = ("The world is a nice place to be in")
string_clear = string.replace(" " ,"")
for character in string_clear:
    ascii_value = ord(character)
    print(f"The ascii value of `{character}` is {ascii_value}")
```

```
The ascii value of `T` is 84
The ascii value of `h` is 104
The ascii value of `e` is 101
The ascii value of `w` is 119
The ascii value of `o` is 111
The ascii value of `r` is 114
The ascii value of `l` is 108
The ascii value of `d` is 100
The ascii value of `i` is 105
The ascii value of `s` is 115
The ascii value of `a` is 97
The ascii value of `n` is 110
The ascii value of `i` is 105
The ascii value of `c` is 99
The ascii value of `e` is 101
The ascii value of `p` is 112
The ascii value of `l` is 108
The ascii value of `a` is 97
The ascii value of `c` is 99
The ascii value of `e` is 101
The ascii value of `t` is 116
The ascii value of `o` is 111
The ascii value of `b` is 98
The ascii value of `e` is 101
The ascii value of `i` is 105
The ascii value of `n` is 110
```

Q26. Convert a decimal number to binary using loops.

- Divide the decimal number by 2.
- down the remainder (0 or 1).
- Use the quotient for the next division.
- Repeat steps 1–3 until the quotient becomes 0.
- Binary number = remainders read in reverse order (from last to first).

In [9]:
```python
number = int(input("ENTER A DECIMAL NUMBER"))
original_number = number
binary = ""

while number > 0:
    reminder = number % 2
    binary = str(reminder) + binary
    number = number // 2
print(f"The binary of {original_number} is {binary}")
```

```
The binary of 13 is 1101
```

Q27. Create a program to find the square root of a number.

In [11]:
```
import math
number = int(input("Enter the number you want to find sq root of"))
x = math.sqrt(25)
print(f"The sq root of {number} is {x}")
```

```
The sq root of 25 is 5.0
```

In [18]:
```
 # Newton's Method (Iterative approach)
number = float(input("Enter a number :"))
guess = number / 2
for i in range (20): # increase the count to increase the efficinecy
    guess = (guess + number / guess) / 2

print(f"Square root of {number} is {guess} approximately")
```

```
Square root of 62500.0 is 250.0 approximately
```

Q28. Write a program to find the sum of all even numbers in a list.

In [24]:
```
a = [1,2,3,4,5,6,78,8,1,22,34,2,9]
the_sum = 0
for numbers in a:
    if int(numbers)%2 == 0 :
        the_sum = the_sum + numbers
    else:
        pass
print(f"The sum of the even numbers of the list is {the_sum}")
```

```
The sum of the even numbers of the list is 156
```

Q29. Create a program to check whether a number is prime or not.

In [30]:
```
number = int(input("Enter a number"))
div_counter = 0
for i in range (1,number+1):
    if number % i == 0:
        div_counter += 1
if div_counter == 2:
    print(f"{number} is a prime number")
else:
    print(f"{number} is not a prime number")
```

```
5 is a prime number
```

Q30. Write a program to display the cube of the number up to an integer.

In [32]:
```
number = int(input("Enter a number"))
cube_of_number = number**3
print(cube_of_number)
```

```
125
```

In [5]:
```
import math
number = int(input("Enter a number"))
cube = math.pow(number,3)
print(cube)
```

```
729.0
```

In [9]:
```
integer = 9
for i in range(1,integer+1):
    print(f"The cube of {i} is {i**3}")
```

```
The cube of 1 is 1
The cube of 2 is 8
The cube of 3 is 27
The cube of 4 is 64
The cube of 5 is 125
The cube of 6 is 216
The cube of 7 is 343
The cube of 8 is 512
The cube of 9 is 729
```

Week 2: Functions, Lists, Tuples, Dictionaries, Sets

1. Write a function to check if a number is even.

In [1]:
```
def odd_even_checker(x):
    if x % 2 == 0 :
        print(f"{x} is an even number")
    else :
        print(f"{x} is a odd number")
odd_even_checker(10)
odd_even_checker(3)
```

```
10 is an even number
3 is a odd number
```

2. Create a list and find the sum of all its elements.

In [4]:
```
a = [1,2,3,4,5,7,8,9,90]
total = 0
for digits in a:
    total = total + int(digits)
print(f"{total} is the sum of all the elements of the list {a}")
```

```
129 is the sum of all the elements of the list [1, 2, 3, 4, 5, 7, 8, 9, 90]
```

3. Write a program to find the maximum and minimum in a list.

In [11]:
```
a = [1,2,3,45,33,67,89,100]
x = int(a[0])

# for finding the maximum number in the list

for digits in a :
    if int(digits) > x :
        x = int(digits)
print(f"{x} is the maximum number in the list")

# for finding the minimum number in the list

y = a[0]
for digits in a :
    if int(digits) < y :
        y = int(digit)
print(f"{y} is the minimum number in the list")
```

```
100 is the maximum number in the list
1 is the minimum number in the list
```

4. Create a program that removes duplicates from a list.

In [17]:
```
a = [1,2,3,4,5,6,2,4,6,1,9,0,2,5,7,2,3,5]
b = []
for items in a :
    if items not in b:
        b.append(items)
print(b)
```

```
[1, 2, 3, 4, 5, 6, 9, 0, 7]
```

In [18]:
```
a = [1,2,3,4,5,6,2,4,6,1,9,0,2,5,7,2,3,5]

for i in range(len(a)):
    j = len(a)-1
    while j > i:
        if a[i] == a[j]:
            a.pop(j)
        j -= 1
print(a)
```

```
[1, 2, 3, 4, 5, 6, 9, 0, 7]
```

5. Write a function to reverse a list.

In [20]:
```
def rev_list(x):
    print(x[::-1])
rev_list([1,2,3,4,5,6,7,8,9])
```

```
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

6. Create a tuple and access its elements.

In [33]:
```
a = ("ram","hari","sita",1,2,3,4,9.0)
print(type(a))
for i in range (len(a)):
    print(f"{a[i]} is the item with index {i} in the list")
```

```
<class 'tuple'>
ram is the item with index 0 in the list
hari is the item with index 1 in the list
sita is the item with index 2 in the list
1 is the item with index 3 in the list
2 is the item with index 4 in the list
3 is the item with index 5 in the list
4 is the item with index 6 in the list
9.0 is the item with index 7 in the list
```

7. Convert a list into a tuple and vice versa.

In [38]:
```
a = [1,2,3,4,5]
```

```
a = tuple(a)
print(type(a))
print("a is now converted to tuple",a)
b = (12,3,4,5,6)
b = list(b)
print(type(b))
print("b is now converted to list",b)
```

```
<class 'tuple'>
a is now converted to tuple (1, 2, 3, 4, 5)
<class 'list'>
b is now converted to list [12, 3, 4, 5, 6]
```

8. Write a program to merge two dictionaries.

In [41]:
```
dict_1 = {"ram":10,"hari":20,"sita":13}
dict_2 = {"reeta":12,"sandhya":26,"sanggeta":34}
print({**dict_1, **dict_2})
```

```
{'ram': 10, 'hari': 20, 'sita': 13, 'reeta': 12, 'sandhya': 26, 'sanggeta': 34}
```

9. Write a function to count the frequency of elements in a list.

In [42]:
```
def count_frequency(lst):
    freq = {}
    for item in lst:
        if item in freq:
            freq[item] += 1
        else:
            freq[item] = 1
    return freq

my_list = [1,2,3,4,5,6,7,1,2,4,7,3,4]
print(count_frequency(my_list))
```

```
{1: 2, 2: 2, 3: 2, 4: 3, 5: 1, 6: 1, 7: 2}
```

10. Create a dictionary of squares of numbers from 1 to 10.

In [48]:
```
square = {}
for i in range(1,11):
    square[i] = i**2
print(square)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

11. Write a program to sort a list in ascending order.

In [64]:
```
a = [1,4,3,2,9,8,7,6,55,11,45,33]
```

```
sorted_list = sorted(a)
print(a)
print(sorted_list)
```

```
[1, 4, 3, 2, 9, 8, 7, 6, 55, 11, 45, 33]
[1, 2, 3, 4, 6, 7, 8, 9, 11, 33, 45, 55]
```

In [66]:
```
a = [1,4,3,2,9,8,7,6,55,11,45,33]

# Bubble sort
for i in range(len(a)):
    for j in range(0, len(a)-i-1):
        if a[j] > a[j+1]:
            a[j],a[j+1] = a[j+1],a[j]

print(a)
```

```
[1, 2, 3, 4, 6, 7, 8, 9, 11, 33, 45, 55]
```

In [67]:
```
a = [1, 4, 3, 2, 9, 8, 7, 6, 55, 11, 45, 33]

# Selection sort
for i in range(len(a)):
    min_index = i
    for j in range(i + 1, len(a)):
        if a[j] < a[min_index]:
            min_index = j
    # Swap the found minimum element with the first element
    a[i], a[min_index] = a[min_index], a[i]

print(a)
```

```
[1, 2, 3, 4, 6, 7, 8, 9, 11, 33, 45, 55]
```

12. Create a program to check if a key exists in a dictionary.

In [86]:
```
d = {"ram":11,"hari":12,"sita":13}

key_to_check = "ram"

# Check if key exists
if key_to_check in d:
    print(f"The key '{key_to_check}' exists in the dictionary.")
else:
    print(f"The key '{key_to_check}' does not exist in the dictionary.")
```

```
The key 'ram' exists in the dictionary.
```

13. Create a set and perform union, intersection, and difference.

In [90]:
```
s1 = {1,2,3,4,56,7}
```

```
s2 = {2,4,6,8,10,12}
print(s1 or s2) # union
print(s1 and s2) # intersection
print(s1 - s2) # difference
print(s1.difference(s2)) # difference
```

```
{1, 2, 3, 4, 7, 56}
{2, 4, 6, 8, 10, 12}
{56, 1, 3, 7}
{56, 1, 3, 7}
```

14. Write a function to find common elements in two lists.

In [93]:
```
l1 = [1,2,3,4,5,7,8]
l2 = [2,5,6,78,3,2]
for item in l1:
    if item in l2:
        print(f"{item} is a comman element in two lists")
```

```
2 is a comman element in two lists
3 is a comman element in two lists
5 is a comman element in two lists
```

15. Write a function that returns the factorial of a number.

In [98]:
```
def factorial(n):
    fact = 1
    x = n
    for i in range(1,n+1):
        fact = fact * i
    print(f"The factorial of {x} is {fact}")
factorial(5)
```

```
The factorial of 5 is 120
```

16. Create a function that checks whether a string is a palindrome.

In [101…
```
def palindrome(string):
    x = string
    y = x[::-1]
    if x == y :
        print("palindrome")
    else :
        print("not palindrome")
palindrome("dad")
palindrome("mama")
```

```
palindrome
not palindrome
```

17. Write a function to count vowels in a string.

In [107…
```python
def vowel_counter(string):
    vowel = "aeiou"
    vowel_counter = 0
    x = string.lower().replace(" ","")
    for characters in x:
        if characters in vowel:
            vowel_counter += 1
    print(f"There are {vowel_counter} vowel in the string")


vowel_counter(" HEY there what is going on")
```

```
There are 8 vowel in the string
```

18. Create a dictionary and iterate over its keys and values.

In [120…
```python
my_dict = {"a": 1, "b": 2, "c": 3}

for i in my_dict:
    print("the keys in the dictionary are :",i)

print("\n")

for i in my_dict:
    print("the values in the dictionary are :",str(my_dict[i]))
```

```
the keys in the dictionary are : a
the keys in the dictionary are : b
the keys in the dictionary are : c


the values in the dictionary are : 1
the values in the dictionary are : 2
the values in the dictionary are : 3
```

19. Write a function to remove all punctuation from a string.

In [125…
```python
def punc_remover(text):
    punc = """!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~"""
    result = ""
    for character in text:
        if character not in punc:
            result += character
    print(result)

punc_remover("hello world, what's up!!! how you'll doing.")
```

```
hello world whats up how youll doing
```

20. Write a function to capitalize the first letter of each word in a string.

In [132…
```
def capitalizer(string):
    x = string
    x = x.title()
    print(x)
capitalizer("hello there hope you are doing well")
```

```
Hello There Hope You Are Doing Well
```

21. Create a list comprehension to get squares of all even numbers in a range.

In [146…
```
squares = [f"the square of {x} is: {x**2}" for x in range(1, 10)]

for item in squares:
    print(item)
```

```
the square of 1 is: 1
the square of 2 is: 4
the square of 3 is: 9
the square of 4 is: 16
the square of 5 is: 25
the square of 6 is: 36
the square of 7 is: 49
the square of 8 is: 64
the square of 9 is: 81
```

22. Write a function to check if a string is an anagram.

In [155…
```
def anagram_checker(x, y):
    # normalize: lowercase and remove spaces
    a = x.lower().replace(" ", "")
    b = y.lower().replace(" ", "")

    # check if sorted characters are equal
    if sorted(a) == sorted(b):
        print("Anagram")
    else:
        print("Not anagram")

# Test cases
anagram_checker("listen", "silent")       #  Anagram
anagram_checker("evil", "vile")           #  Anagram
anagram_checker("dormitory", "dirty room")#  Anagram
anagram_checker("hero", "zero")           #  Not anagram
anagram_checker("aaa", "abcaaa")          #  Not anagram
```

```
Anagram
Anagram
Anagram
Not anagram
Not anagram
```

23. Create a nested dictionary to represent student records.

In [158…
```python
dict1 = {"name":"swopnim","age":23}
dict2 = {"class":12,"school":"Everest"}
nested_dict =  {"Student_details":dict1,"School_detail":dict2}
print(nested_dict)
```

```
{'Student_details': {'name': 'swopnim', 'age': 23}, 'School_detail': {'class':
12, 'school': 'Everest'}}
```

24. Write a function to flatten a nested list.

In [165…
```python
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))  # recursive call
        else:
            result.append(item)
    return result

nested = [1, 2, [3, 4], [5, [6, 7]]]
print(flatten(nested))
```

```
[1, 2, 3, 4, 5, 6, 7]
```

In [ ]:
```python

```

25. Write a program to find the second highest number in a list.

In [172…
```python
a = [1,23,12,45,22,19,90,76,43,22,87]
print("unsorted list",a)
sorted_list = sorted(a)
print("sorted list",sorted_list)
print("The second highest number is : ",sorted_list[-2])
```

```
unsorted list [1, 23, 12, 45, 22, 19, 90, 76, 43, 22, 87]
sorted list [1, 12, 19, 22, 22, 23, 43, 45, 76, 87, 90]
The second highest number is :  87
```

26. Create a function to rotate a list left by k positions.

In [178…
```
k = 7
a = [1,2,3,4,5]
b = []
c = []
if k > len(a):
    k = k % len(a)
for i in range (k):
    b.append(a[i])
print(b)
for i in range(k,len(a)):
    c.append(a[i])
print(c)
d = c + b
print(d)
```

```
[1, 2]
[3, 4, 5]
[3, 4, 5, 1, 2]
```

In [179…
```
k = 7
a = [1,2,3,4,5]
if k > len(a):
    k = k % len(a)

first_part = a[:k]
second_part = a[k:]
rotated_list = second_part + first_part
print(rotated_list)
```

```
[3, 4, 5, 1, 2]
```

27. Write a function to find the missing number from a list of 1 to N.

In [182…
```
our_list = [1,2,3,5,7,8,10]
computer_list = []
n = int(input("enter the range"))
for i in range (1,n+1):
    computer_list.append(i)
for nums in computer_list:
    if nums not in our_list:
        print(f"{nums} is a missing number in our list")
```

```
4 is a missing number in our list
6 is a missing number in our list
9 is a missing number in our list
```

In [183…
```
def find_missing(our_list, n):
    for i in range(1, n+1):
        if i not in our_list:
            print(f"{i} is missing")
```

```
find_missing([1,2,3,5,7,8,10],10)
```

```
4 is missing
6 is missing
9 is missing
```

28. Write a program to remove all None values from a list.

In [189…
```
my_list = [1, None, 3, 0, "",[], False]
b = []
for items in my_list:
    if items is not None:
        b.append(items)
print(b)
```

```
[1, 3, 0, '', [], False]
```

In [191…
```
my_list = [1, None, 3, 0, "",[], False]
b = [item for item in my_list if item is not None]
print(b)
```

```
[1, 3, 0, '', [], False]
```

29. Write a function to merge two dictionaries and handle key collisions by summing values.

In [199…
```
def merge_dicts_sum(dict1,dict2):
    merged = dict1.copy()
    for key,value in dict2.items():
        if key in merged:
            merged[key] += value
        else:
            merged[key] = value
    return merged

dict1 = {"a": 1, "b": 2, "c": 5}
dict2 = {"b": 3, "c": 4, "d": 7, "e":10}

result = merge_dicts_sum(dict1, dict2)
print(result)
```

```
{'a': 1, 'b': 5, 'c': 9, 'd': 7, 'e': 10}
```

30. Create a function to find unique elements present in only one of two lists.

In [202…
```
list1 = [1,2,3,4,5,6,7]
list2 = [5,6,7,8,9,0]
list1unq = []
list2unq = []
for i in list1:
```

```
    if i not in list2:
        list1unq.append(i)

for i in list2:
    if i not in list1:
        list2unq.append(i)

reslist = list1unq + list2unq
print(reslist)
```

```
[1, 2, 3, 4, 8, 9, 0]
```

In [205…

```
# A ^ B = (A - B) ∪ (B - A)

reslist1 = list(set(list1) ^ set(list2))
print(reslist1)
```

```
[0, 1, 2, 3, 4, 8, 9]
```

Week 3: File Handling, Error Handling, Modules, Comprehensions

Q1. Write a Python script to read a file and print its contents.

In [18]:
```
# we already have a file named practise.txt here we open the file and read it
# This will get the job done but isn't standard and optimum
file = open("practise.txt","r")
file.read()
```

Out[18]:
```
'The quiet rhythm of early morning often sets the tone for the entire day. As
the first rays of sunlight filter through the windows, the world outside slow
ly begins to stir with life. Birds call to one another, the air feels crisp,
and there is a sense of calm clarity. People who rise early often find these
hours to be the most productive and peaceful, free from distractions and nois
e. Whether used for reflection, study, or simply savoring a warm cup of tea,
mornings carry a unique energy that inspires focus, creativity, and a gentle
appreciation for simple beginnings.'
```

In [5]:
```
# Improvised Method 1:
with open("practise.txt","r") as file:
    contents = file.read()
    print(contents)
file.close()
```

```
The quiet rhythm of early morning often sets the tone for the entire day. As th
e first rays of sunlight filter through the windows, the world outside slowly b
egins to stir with life. Birds call to one another, the air feels crisp, and th
ere is a sense of calm clarity. People who rise early often find these hours to
be the most productive and peaceful, free from distractions and noise. Whether
used for reflection, study, or simply savoring a warm cup of tea, mornings carr
y a unique energy that inspires focus, creativity, and a gentle appreciation fo
r simple beginnings.
```

In [6]:
```
# Imporvised Method 2:
file = open("practise.txt","r")
contents = file.read()
print(contents)
file.close()
```

```
The quiet rhythm of early morning often sets the tone for the entire day. As th
e first rays of sunlight filter through the windows, the world outside slowly b
egins to stir with life. Birds call to one another, the air feels crisp, and th
ere is a sense of calm clarity. People who rise early often find these hours to
be the most productive and peaceful, free from distractions and noise. Whether
used for reflection, study, or simply savoring a warm cup of tea, mornings carr
y a unique energy that inspires focus, creativity, and a gentle appreciation fo
r simple beginnings.
```

Q2. Create a file and write your name into it.

In [13]:
```
file = open("swopnim.txt","w")
contents = file.write("Hello everyone , welcome to this notebook")
file.close()
```

In [14]:
```
file = open("swopnim.txt","r")
contents = file.read()
print(contents)
file.close()
```

```
Hello everyone , welcome to this notebook
```

In [16]:
```
# Cleaner way

# Writing into a file
with open("swopnim.txt", "w") as file:
    file.write("Hello everyone, welcome to this notebook")

# Reading the file
with open("swopnim.txt", "r") as file:
    print(file.read())

file.close()
```

```
Hello everyone, welcome to this notebook
```

Q3. Handle a ZeroDivisionError using try-except.

In [26]:
```
def div(a,b):
    try:
        return a / b
    except ZeroDivisionError:
        print("You have ran into ZeroDivisionError")

print(div(2,3))
print(div(3,0))
```

```
0.6666666666666666
You have ran into ZeroDivisionError
None
```

Why `None` in the second case?

Because inside the `except` block, you only printed a message and didn't `return` anything. In Python, when a function doesn't return a value, it automatically returns `None`.

In [27]:
```
def div(a, b):
    try:
        return a / b
```

```
    except ZeroDivisionError:
        return "Division by zero is not allowed!"

print(div(2, 3))  # 0.666...
print(div(3, 0))  # "Division by zero is not allowed!"
```

```
0.6666666666666666
Division by zero is not allowed!
```

Q4. Write a program to handle file not found error.

In [29]:
```
# This gets the job done but improvised version is below
def file_reader():
    try:
        file = open("xyz.txt","r")
    except FileNotFoundError:
        return "file not found error"

file_reader()
```

Out[29]:
```
'file not found error'
```

In [32]:
```
def file_reader():
    try:
        with open("xyz.txt","r") as file:
            contents = file.read()
            return contents

    except FileNotFoundError:
        return "FileNotFoundError"

print(file_reader())
```

```
FileNotFoundError
```

Q5. Create a module with a function and import it in another file.

In [51]:
```
import addition
import importlib
importlib.reload(addition)

print(addition.add_numbers(10, 20))
```

```
30
```

Q6. Use a list comprehension to filter even numbers from a list.

In [60]:
```
l = [1,2,3,4,5,6,7,8,9,10]
new_l = [ i for i in l if i % 2 == 0  ]
```

```
print(new_l)
```

```
[2, 4, 6, 8, 10]
```

Q7. Write a generator that yields even numbers up to N.

In [69]:
```
def even_number(n):
    for i in range(2,n+1,2):
        yield i

# Example usage
for num in even_number(10):
    print(num)

print("\n")

for items in even_number(20):
    print(items)
```

```
2
4
6
8
10


2
4
6
8
10
12
14
16
18
20
```

In [64]:
```
def even_number_list(n):
    return [i for i in range(2,n+1,2)]

print(even_number_list(10))
```

```
[2, 4, 6, 8, 10]
```

Q8. Create a program to count lines and words in a file.

In [74]:
```
file_path = "Practise.txt"

lines_count = 0
words_count = 0

with open(file_path,"r") as f:
    for line in f:
```

```
        lines_count += 1
        words_count += len(line.split()) # split line into words and count

print(f"Number of lines: {lines_count}")
print(f"Number of words: {words_count}")
```

```
Number of lines: 13
Number of words: 99
```

Q9. Write a program to read a CSV file and print its contents.

In [77]:
```
# Method 1:

# writing a csv file
import csv

# Data to write
data = [
    ["Name", "Age", "City"],
    ["Alice", 25, "New York"],
    ["Bob", 30, "London"],
    ["Charlie", 22, "Paris"]
]

# Writing to CSV
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data)  # writes all rows at once

print("CSV file written successfully!")
```

```
CSV file written successfully!
```

In [78]:
```
import csv

with open("output.csv","r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

```
['Name', 'Age', 'City']
['Alice', '25', 'New York']
['Bob', '30', 'London']
['Charlie', '22', 'Paris']
```

In [79]:
```
# Method 2:

import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 22],
```

```
    "City": ["New York", "London", "Paris"]
})

# Write to CSV
df.to_csv("output.csv", index=False)  # index=False avoids writing row numbers

print("CSV file written successfully!")
```

```
CSV file written successfully!
```

In [80]:
```
import pandas as pd

df = pd.read_csv("output.csv")
print(df)
```

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30    London
2  Charlie   22     Paris
```

Q10. Handle multiple exceptions in a single try block.

In [ ]:
```
def test(a,b):
    try:
        return a+b
    except :
        return a/b
    except
```

Week 4: OOPs Concepts

Q1. Write a Python program to check if a string has all unique characters.

In [2]:
```python
# set removes dublicate values
def has_unique_chars(s):
    return len(s) == len(set(s))

# Examples
print(has_unique_chars("Python"))
print(has_unique_chars("hello"))
```

```
True
False
```

In [7]:
```python
def has_unique_chars(s):
    seen = set(s)
    for characters in s:
        if characters in seen:
            return True
        seen.add(character)
    return False

print(has_unique_chars("python"))
print(has_unique_chars("Hello"))
```

```
True
True
```

Q2. Create a program that removes all duplicate characters from a string.

In [12]:
```python
# Wrong appraoch
def remove_duplicate(s):
    return set(s)
remove_duplicate("hello")
```

Out[12]:
```
{'e', 'h', 'l', 'o'}
```

In [14]:
```python
# Right appraoch
def remove_duplicate(s):
    result = ""
    for char in s:
        if char not in result:
            result += char
    return result
print(remove_duplicate("hello"))
```

```
helo
```

In [17]:
```
# alternate appraoch using dictionary
# dict.fromkeys() removes duplicates while keeping the order — super clean and
def remove_duplicate(s):
    return ''.join(dict.fromkeys(s))

print(remove_duplicate("hello"))  # helo
```

```
helo
```

Q3. Write a script to count the frequency of each character in a string.

In [7]:
```
text = "hello world"

char_frequency = {}

for characters in text:
    if characters in char_frequency:
        char_frequency[characters] += 1
    else:
        char_frequency[characters] = 1

print(char_frequency)

print("\n")

for char, count in char_frequency.items():
    print(f"'{char}' : {count}")
```

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}


'h' : 1
'e' : 1
'l' : 3
'o' : 2
' ' : 1
'w' : 1
'r' : 1
'd' : 1
```

Q4. Write a program that accepts a sentence and calculates the number of upper and lower case letters.

In [9]:
```
sentence = "Her There"

upper_count = 0
lower_count = 0

for characters in sentence :
    if characters.isupper():
        upper_count += 1
```

```
    elif characters.islower():
        lower_count += 1

print("No of uppercase letters is: ",upper_count)
print("No of lowercase letters is: ",lower_count)
```

```
No of uppercase letters is:  2
No of lowercase letters is:  6
```

Q5. Create a program to find the longest word in a sentence.

In [10]:
```
sentence = " The world is full of people that want to controll others"

words = sentence.split()

longest_word = ""

for word in words :
    if len(word) > len(longest_word):
        longest_word = word

print("The longest word is :", longest_word)
```

```
The longest word is : controll
```

Q6. Write a program that takes a string and returns the string in reverse order without using [::-1].

In [18]:
```
string = "hey world what is going on"
reversed_string = ""

for i in string:
    reversed_string = i + reversed_string

print(reversed_string)
```

```
no gniog si tahw dlrow yeh
```

In [31]:
```
string = "hey world what is going on"
char_list = list(string)

n = len(char_list)

for i in range (n//2):
    char_list[i], char_list[n - i -1] = char_list[n - i - 1], char_list[i]

new_string = "".join(char_list)
print(new_string)
```

```
no gniog si tahw dlrow yeh
```

Q7. Create a Python function to check if a string is a pangram.

In [37]:
```
import string
def is_pangram(sentence):
    sentence = sentence.lower()
    for letter in string.ascii_lowercase:
        if letter not in sentence :
            return False
    return True

print(is_pangram("The quick brown fox jumps over the lazy dog."))
print(is_pangram("HI there "))
```

```
True
False
```

Q8. Write a Python script to sort words in a sentence alphabetically

In [45]:
```
# this does not respect the upper case so if upper case is introduced it'll ke
sentence = "What are the things that you should value in life"

words = sentence.split()

words.sort()

sorted_sentence = " ".join(words)

print(sorted_sentence)
```

```
What are in life should that the things value you
```

In [46]:
```
# Fixing the above problem
sentence = "What are the things that you should value in life"

words = sentence.split()

words.sort(key=str.lower)

sorted_sentence = " ".join(words)

print(sorted_sentence)
```

```
are in life should that the things value What you
```

In [47]:
```
sentence = "What are the things that you should value in life"

sentence = sentence.lower()

words = sentence.split()
```

```
words.sort()

sorted_sentence = " ".join(words)

print(sorted_sentence)
```

```
are in life should that the things value what you
```

Q9. Write a program to check if two strings are anagrams

In [52]:
```
def check(string1,string2):
    string1 = string1.replace(" ","").lower()
    string2 = string2.replace(" ","").lower()

    return sorted(string1) == sorted(string2)

print(check("listen","silent"))
print(check("hi","hey"))
```

```
True
False
```

Q10. Write a Python program to capitalize the first letter of each word in a sentence.

In [55]:
```
sentence = "hey there what is going on with you"

capitalized_sentence = sentence.title()

print("capitalised_sentence : ",capitalized_sentence)
```

```
capitalised_sentence :  Hey There What Is Going On With You
```

In [58]:
```
sentence = "hey there what is going on with you"
words = sentence.split()

capitalized_words = [word.capitalize() for word in words]

capitalized_sentence = " ".join(capitalized_words)

print("capitalized sentence :", capitalized_sentence)
```

```
capitalized sentence : Hey There What Is Going On With You
```

Q11. Create a program that extracts numbers from a string and returns their sum.

In [61]:
```
number_in_string = "12345"
total = 0

for i in number_in_string:
    total = total + int(i)
```

```
print(total)
```

```
15
```

Q12. Write a program to replace all spaces in a string with underscores.

In [63]:
```
string = "hello everyone i just want to emphasise on the fact that no matter w
string = string.replace(" ","_")
print(string)
```

```
hello_everyone_i_just_want_to_emphasise_on_the_fact_that_no_matter_what_you_sho
uld_work_hard
```

Q13. Write a function to count how many times a substring appears in a string.

In [64]:
```
def count_substring(string,substring):
    # using a build in function
    return string.count(substring)

count_substring("the world , the mountain , the hero","the")
```

Out[64]:
```
3
```

Q14. Write a script to convert a string into title case without using .title().

In [72]:
```
def to_title_case(sentence):
    words = sentence.split()
    title_words = []

    for word in words :
        if len(word) > 0:
            first_char = word[0]
            if 'a' <= first_char <= 'z':
                first_char = chr(ord(first_char)-32)

            new_word = first_char + word[1:]
            title_words.append(new_word)
        else:
            title_words.append(word)

    return " ".join(title_words)
print(to_title_case("hello eveyone how are you doing"))
```

```
Hello Eveyone How Are You Doing
```

Q15. Write a Python program to merge two dictionaries into one.

In [73]:
```
def merge_dict (dict1, dict2):
    merge = {}

    for key in dict1:
        merge[key] = dict1[key]

    for key in dict2:
        merge[key] = dict2[key]

    return merge

print(merge_dict({"a":1,"b":2,"c":3},{"d":4,"e":5,"f":6}))
```

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6}
```

Q16. Create a program to filter out all non-alphabetic characters from a string.

In [79]:
```
def filter_aplha(string):
    result = ""
    for char in string:
        if('a' <= char <= 'z') or ('A' <= char <= 'Z'):
            result += char
    return result

print(filter_aplha("hello , WOrld!!! 123 Py000thon #201"))
```

```
helloWOrldPython
```

Q17. Write a function that returns True if a string ends with a given suffix.

In [81]:
```
def ends_with(string,suffix):
    str_len = len(string)
    suf_len = len(suffix)

    if suf_len > str_len:
        return False

    for i in range(1, suf_len + 1):
        if string[-i] != suffix[-i]:
            return False
    return True

print(ends_with("development","ment"))
print(ends_with("hello","Hi"))
```

```
True
False
```

Q18. Create a program that counts words, characters, and lines in a paragraph.

In [82]:
```
paragraph = "hello mate how are you doing in life"

line_count = paragraph.count("\n") + 1 # Count "\n" and add 1

word_count = len(paragraph.split())

char_count = len(paragraph)

print("Lines:", line_count)
print("Words:", word_count)
print("Characters:", char_count)
```

```
Lines: 1
Words: 8
Characters: 36
```

Q19. Write a script to encode a string using Caesar cipher (shift = 3).

What the question is asking:

1. Input: A string of text from the user.
   - Example: `"Hello World"`
2. Task: Encode the string using a Caesar cipher with a shift of 3.
   - A Caesar cipher is a type of substitution cipher where each letter is replaced by another letter a fixed number of positions down the alphabet.
   - Here, the shift = 3 means:
     - A → D, B → E, C → F …
     - X → A, Y → B, Z → C (wraps around at the end of the alphabet)
3. Output: The new encoded string.

Example: `"Hello World"` → `"Khoor Zruog"`

In [85]:
```
def caser_cipher(text , shift = 3):
    result = ""

    for char in text:
```

```
        # Check if character is uppercase
        if 'A' <= char <= 'Z':
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))

        # Check if character is lowercase
        elif 'a' <= char<= 'z':
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))

        else:
            result += char

    return result

print(caser_cipher("Hey mate what is going on with you"))
```

```
Khb pdwh zkdw lv jrlqj rq zlwk brx
```

Q20. Write a program that accepts a string and counts vowels and consonants.

In [89]:
```
def count_vowel_consonant(text):
    vowels = "aeiouAEIOU"  # include uppercase vowels
    vowel_count = 0
    consonant_count = 0

    for char in text:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

print(count_vowel_consonant("hey mate what's going on in your life?"))
```

```
(12, 17)
```

Q21. Create a script to convert binary string to decimal.

In [90]:
```
def binary_to_decimal(binary_str):
    decimal = 0
    power = 0 # start from rightmost big

    # process the binary string from right to left
    for digit in reversed(binary_str):
        if digit == "1":
            decimal += 2 ** power
        elif digit!= "0":
            return "Error: Not a valid binary number"
        power += 1
```

```
    return decimal

print(binary_to_decimal("1010111"))
```

```
87
```

Q22. Write a program to count the number of words starting with a vowel in a string.

In [92]:
```
def count_words_starting_with_vowel(text):
    vowel = "aeiouAEIOU"
    count = 0
    words = text.split()
    for word in words:
        if len(word) > 0 and word[0] in vowel :
            count += 1
    return count

print(count_words_starting_with_vowel("apple egg what is it"))
```

```
4
```

Q23. Create a script that takes a sentence and removes all stop words.

In [98]:
```
def remove_stop_words(x):
    stop_words = {
        "a", "an", "the", "is", "in", "on", "at", "for", "of",
        "and", "or", "to", "with", "by", "this", "that", "are"
    }

    word = x.split()
    filter_words = [word for word in words if word.lower() not in stop_words]
    return " ".join(filter_words)

print(remove_stop_words("This is a simple example for learning Python with cla
```

```
hey there what going you
```

Q24. Write a Python program to split a sentence into words and reverse each word

In [100…
```
def split_and_reverse(text):
    words = text.split()
    new = []
    for word in words:
        new.append(word[::-1])
    return " ".join(new)

print(split_and_reverse("hey mate what are you doing"))
```

```
yeh etam tahw era uoy gniod
```

Q25. Write a function that returns a new string made of every third character of the original string

In [102…
```
def every_third_char(text):
    result = ""
    for i in range(2,len(text),3):
        result += text[i]
    return result

print(every_third_char("adfflkajdfklajdfklafdlakjd"))
```

```
fkdldldk
```

Q26. Write a program to find all palindromic substrings in a string

In [106…
```
def palindrom_in_string(text):
    words = text.split()
    palindromic_words = []
    for word in words:
        if len(word)>1 and word == word[::-1]:
            palindromic_words.append(word)
    return palindromic_words

print(palindrom_in_string(" madam is going aba in racecar i wish you the best
```

```
['madam', 'aba', 'racecar']
```

In [107…
```
def palindromic_substrings(text):
    palindromes = []
    n = len(text)

    # Loop through all possible substrings
    for i in range(n):
        for j in range(i + 2, n + 1):  # substrings of length >= 2
            substring = text[i:j]
            if substring == substring[::-1]:
                palindromes.append(substring)

    return palindromes


# Example usage
text = "madam is going aba in racecar"
result = palindromic_substrings(text)
print("Palindromic substrings:", result)
```

```
Palindromic substrings: ['madam', 'ada', ' aba ', 'aba', 'racecar', 'aceca', 'c
ec']
```

Q27. Write a function that compresses a string using run-length encoding.

In [109…

```
def run_length_encode(text):
    if not text:
        return ""
    result = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i-1]:
            count += 1
        else:
            result += text[i-1] + str(count)
            count = 1
    result += text[-1] + str(count)
    return result

print(run_length_encode("aaaaaabbbbbbbcccddssee"))
```

```
a6b7c3d2s2e2
```

Q28. Write a Python program to count the frequency of each word in a file.

In [16]:

```
""" 1. Open the file in read mode
    2. Read it's content and plit it into words
    3. use a dictonary to store word frequency
    4. loop through each word:
        - convert it all to lower case
        - remove punctuation if necessary
        - increament of the frequency in the dictonary
"""
def word_frequency(file_name):
    freq = {}
    with open(file_name,"r") as file:
        text = file.read().lower()
    words = text.split()
    for word in words:
        clear_word = ""
        for char in word:
            if char.isaplha():
                clear_word += char
        if clear_word in freq:
            freq[clear_word] += 1
        else:
            freq[clear_word] = 1
    return freq
# You should use a valid file and verify it yoursel just once , check it
print(word_frequency("data.txt"))
```

Q29. Write a script that extracts hashtags from a tweet.

In [110…
```python
def extract_hashtags(tweet):
    hashtag = []

    words = tweet.split()

    for word in words:
        if word.startswith("#") and len(word)>1 :
            hashtag.append(word)
    return hashtag

tweet = "Loving the new features in #Python3.11! #coding #100DaysOfCode"
print("Hashtags found:", extract_hashtags(tweet))
```

```
Hashtags found: ['#Python3.11!', '#coding', '#100DaysOfCode']
```

Q30. Write a function to remove punctuation from a string.

In [114…
```python
import string
def remove_punctuation(text):
    # string.punctuation contains all punctuation characters
    result = text.translate(str.maketrans('', '', string.punctuation))
    return result


# Example usage
text = "Hello, world! How's everything going?"
clean_text = remove_punctuation(text)
print("Text without punctuation:", clean_text)
```

```
Text without punctuation: Hello world Hows everything going
```

Q31. Create a program that finds the first non-repeating character in a string.

In [5]:
```python
def first_non_repeating_character(text):
    for character in text:
        if text.count(character) == 1:
            return character
print(first_non_repeating_character("swiss"))
print(first_non_repeating_character("helloh"))
```

```
w
e
```

Q32. Write a script that converts camelCase to snake_case.

In [6]:
```
import re
def camel_to_snake(name):
    # Insert underscore before each capital letter and convert to lowercase
    snake = re.sub(r'([A-Z])',r'_\1',name).lower()

    # Remove the leadin underscore if the word start with a capital
    return snake.lstrip("_")

camel_to_snake("ConvertThisToSnakeCase")
```

Out[6]:
```
'convert_this_to_snake_case'
```

In [8]:
```
# from scratch
def camel_to_snake(name):
    snake_case = ""
    for char in name:
        if char.isupper():
            snake_case += "_" + char.lower()
        else:
            snake_case += char

    if snake_case[0] == "_":
        snake_case = snake_case[1:]
    return snake_case
camel_to_snake("ConvertThisToSnakeCaseTwo")
```

Out[8]:
```
'convert_this_to_snake_case_two'
```

Q33. Write a function to generate acronyms from a sentence.

In [13]:
```
def generate_acronyms(sentence):
    words = sentence.split()
    acronyms = ""

    for word in words:
        if word[0].isalpha():
            acronyms += word[0].upper()
    return acronyms
print(generate_acronyms("National Aeronautics and Space Administration"))
print(generate_acronym("Portable Network Graphics"))
```

```
NAASA
PNG
```

Q34. Write a script to check if a file contains a specific word.

In [ ]:
```
''' You'll open a file in read mode.
    read it's contents
    search for the word in the contents
    return or print whether the word is found or not.
'''
def check_word_in_file(file_name, word):
    found = False
    with open(file_name,"r") as file:
        for line in file:
            if word.lower() in line.lower():
                found = True
                break
    if found:
        print("the word is found")
    else:
        print("file not found")
check_word_in_file("story.txt", "hero")
```

Q35. Write a Python program to find and replace text in a file.

In [ ]:
```
'''Open a file.
    Read all the text.
    Replace a specific word or phrase with another.
    Save the updated content back into the file (or a new file).
    eg: I love java. if we replace java with python i'll be
        I love python.
'''
def find_and_replace(file_name, old_text, new_text):
    with open(filename,"r") as file:
        content = file.read()

    update_content = ""
    i = 0
    while i < len(content):
        # in next cell we'll use a built in method but let's do it from scratc
        if content[i:i+len(old_text)] == old_text:
            updated_content += new_text
            i += len(old_text)
        else:
            update_content += content[i]
            i += 1
    with open(filename,"w") as file:
        file.write(update_content)
    print("Replacement Completed")
find_and_replace("data.txt","old","new")
```

In [ ]:
```
# using a built in function
```

```
def find_and_replace(filename, old_text, new_text):
    with open(filename, "r") as file:
        content = file.read()
    updated_content = content.replace(old_text, new_text)
    with open(filename, "w") as file:
        file.write(updated_content)
    print("Replacement Completed")
find_and_replace("Hello.txt","Ram","Ramesh")
```

Q36. Write a script that checks if all characters in a string are digits.

In [21]:
```
# checking manually
def all_digit(string):
    for char in string:
        if not ("0" <= char <= "9"):
            return False
    return True
print(all_digit("1234564"))
print(all_digit("12dfgh34dd"))
```

```
True
False
```

In [17]:
```
# using built in approach
def all_digit(string):
    return string.isdigit()
print(all_digit("123456"))
print(all_digit("12dfgh34"))
```

```
True
False
```

Q37. Write a program to calculate the average word length in a sentence.

Split the sentence into words.

Count the total number of characters (excluding spaces/punctuation).

Divide by the number of words.

Example: Sentence → "I love Python" Word lengths → 1, 4, 6 Average = (1 + 4 + 6) / 3 = 3.67

In [22]:
```
def average_word_length(sentence):
    words = sentence.split()
    total_length = 0
    word_count = 0

    for word in words :
```

```
        clean_word = ""
        for char in word:
            if char.isalpha():
                clean_word += char
        total_length += len(clean_word)
        word_count += 1

    if word_count == 0 :
        return o
    return total_length / word_count

print(average_word_length("I love Python programming!"))
```

```
5.5
```

Q38. Create a function that removes all HTML tags from a string.

You need to remove all HTML tags (like

,

,
, , etc.) from a string and keep only the readable text.

Example: Input → "

Hello **World**

" Output → "Hello World"

In [23]:
```
import re
def remove_html_tags(text):
    clean_text = re.sub(r'<.*?>','',text)
    return clean_text
print(remove_html_tags("<h1>Hello <b>World</b>! Welcome to <i>Python</i>.</h1>
```

```
Hello World! Welcome to Python.
```

In [24]:
```
def remove_html_tags(text):
    result = ""
    inside_tag = False

    for char in text:
        if char == '<':
            inside_tag = True
        elif char == '>':
            inside_tag = False
        elif not inside_tag:
            result += char
    return result
```

```
print(remove_html_tags("<h1>Hello <b>World</b>! Welcome to <i>Python is great<
```

```
Hello World! Welcome to Python is great.
```

Q39. Write a program to parse a date string and display it in a different format.

In [27]:
```
# manual apprach
def convert_date_format(date_str):
    months = ["January", "February", "March", "April", "May", "June",
              "July", "August", "September", "October", "November", "December"
    year , month , day = date_str.split("-")
    month_name = months[int(month)-1]
    return f"{month_name} {int(day)},{year}"

print(convert_date_format("2065-10-16"))
```

```
October 16,2065
```

In [28]:
```
# Using built in function
from datetime import datetime
def convert_date_format(date_str):
    date_object = datetime.strptime(date_str,"%Y-%m-%d")
    formatted_date = date_object.strftime("%B %d, %Y")
    return formatted_date
print(convert_date_format("2065-10-16"))
```

```
October 16, 2065
```

Q40. Write a script that finds all email addresses in a given text.

In [30]:
```
def extract_emails(text):
    emails = []
    words = text.split()
    for word in words:
        if "@" in word and "." in word:
            emails.append(word.strip(",."))
    return emails
print(extract_emails("My emails are hello@gmail.com and work.mail@outlook.com.
```

```
['hello@gmail.com', 'work.mail@outlook.com']
```

Q41. Write a program that counts the occurrence of each vowel in a paragraph.

In [32]:
```
def count_vowels(paragraph):
    counts = {'a':0 , 'e':0 , 'i':0 , 'o':0 , 'u':0}
    paragraph = paragraph.lower()
    for char in paragraph:
        if char in counts:
            counts[char] += 1
```

```
    return counts
print(count_vowels("Hello world, I love Python. Programming is fun!"))
```

```
{'a': 1, 'e': 2, 'i': 3, 'o': 5, 'u': 1}
```

Q42. Create a function that validates an email address format. explain and work on

A valid email generally has the following structure:

username@domain.extension

Rules:

username can contain letters, numbers, dots, underscores, and some special characters (+, -).

There must be exactly one @ symbol.

domain contains letters, numbers, dots, or hyphens.

extension comes after a dot and is usually 2–6 letters (like .com, .org, .edu).

Example:

Valid: hello.world@example.com

Invalid: hello@.com, user@@example.com, user@domain

In [34]:
```
def validate_email(email):
    if email.count("@") != 1:
        return False
    username, domain_ext = email.split("@")
    # Check usrename is not empty
    if not username:
        return False
    # Domain must have dot
    if "." not in domain_ext:
        return False
    domain, extension = domain_ext.rsplit(".",1)
    if not domain or not extension:
        return False

    if len(extension)<2 or len(extension)> 6:
        return False
    return True

print(validate_email("hello@example.com"))
print(validate_email("user@@mail.com"))
```

```
True
False
```

Q43. Write a script to check if a string is a valid URL.

Rules (basic check):

- Starts with a valid scheme: http://, https://, or ftp://.
- Followed by a domain (like example.com).
- Optional path, query, or fragment.

`Examples:`

Valid: https://www.example.com, http://my-site.org/path

Invalid: htp://example, example.com, ://wrong.com

In [39]:
```
def validate_url(url):
    valid_schemes = ["http://","https://","ftp://"]
    for scheme in valid_schemes:
        if url.startswith(scheme) and url.endswith(".com"):
            return True
    return False
print(validate_url("https://www.example.com"))
print(validate_url("example.com"))
```

```
True
False
```

Q44. Write a program that extracts all integers from a given text.

In [42]:
```
def extract_integers(text):
    numbers = []
    num_str = ""

    for char in text:
        if char.isdigit():
            num_str += char        # build the number
        else:
            if num_str:            # if we reached a non-digit, save the numbe
                numbers.append(int(num_str))
                num_str = ""
    if num_str:                    # append the last number if any
        numbers.append(int(num_str))

    return numbers

# Example usage
text = "I have 2 apples, 15 oranges, and 100 cherries."
print(extract_integers(text))
```

```
[2, 15, 100]
```

Q45. Create a script to find duplicate words in a paragraph.

In [44]:
```python
import string
def find_duplicate_words(paragraph):
    paragraph = paragraph.lower()
    for p in string.punctuation:
        paragraph = paragraph.replace(p,"")

    words = paragraph.split()
    word_count = {}
    duplicates = set()

    for word in words:
        if word in word_count:
            word_count[word] += 1
            duplicates.add(word)
        else:
            word_count[word] = 1
    return duplicates, word_count
print(find_duplicate_words("This is a test. This test is simple."))
```

```
({'test', 'this', 'is'}, {'this': 2, 'is': 2, 'a': 1, 'test': 2, 'simple': 1})
```

Q46. Write a program that converts a sentence to Pig Latin.

Pig Latin is a simple way to transform English words into a playful language. The basic rules:

1. For words that start with a consonant, move all letters before the first vowel to the end, then add "ay".

- Example: "hello" → "ellohay"

2. For words that start with a vowel, just add "way" at the end.

- Example: "apple" → "appleway"

In [45]:
```python
def pig_latin(sentence):
    vowels = "aeiouAEIOU"
    words = sentence.split()
    pig_latin_words = []

    for word in words:
        if word[0] in vowels:
            # Word starts with a vowel
```

```
            pig_word = word + "way"
        else:
            # Word starts with consonant
            # Find the first vowel
            for i, letter in enumerate(word):
                if letter in vowels:
                    pig_word = word[i:] + word[:i] + "ay"
                    break
            else:
                # No vowel found, treat entire word as consonant
                pig_word = word + "ay"
        pig_latin_words.append(pig_word)

    return " ".join(pig_latin_words)

# Example usage
sentence = "hello world I love Python"
print(pig_latin(sentence))
```

```
ellohay orldway Iway ovelay onPythay
```

Q47. Write a script that finds the longest sentence in a paragraph.

In [46]:
```
import re
def longest_sentence_by_words(paragraph):
    sentences = re.split(r'[.!?]\s*', paragraph)
    sentence_length = {}

    for sentence in sentences:
        sentence = sentence.strip()
        if sentence :
            sentence_length[sentence] = len(sentence.split())

    longest = max(sentence_length , key = sentence_length.get)
    return {longest: sentence_length[longest]}

paragraph = "Hello world! This is a test paragraph. It has multiple sentences,
print(longest_sentence_by_words(paragraph))
```

```
{'It has multiple sentences, and we want to find the longest one': 12}
```

Q48. Write a Python program to read a file and display all lines that contain a given keyword.

You need a program that:

Reads a text file line by line.

Checks if a given keyword exists in each line.

Prints only the lines that contain the keyword.

In [ ]:
```python
def find_lines_with_keyword(filename, keyword):
    with open(filename,"w") as file:
        for line in file:
            if keyword in line:
                print(line.strip())

find_lines_with_keyword("example.txt","python")
```

Q49. Write a script to clean a text file by removing extra spaces and blank lines

In [ ]:
```python
def clean_text_file(filename):
    with open(filename, 'r') as file:
        lines = file.readlines()

    cleaned_lines = []
    for line in lines:
        # Remove leading/trailing spaces and reduce multiple spaces to single
        cleaned_line = ' '.join(line.strip().split())
        if cleaned_line:  # ignore empty lines
            cleaned_lines.append(cleaned_line)

    # Optionally, overwrite the file with cleaned content
    with open(filename, 'w') as file:
        for line in cleaned_lines:
            file.write(line + '\n')

    print("File cleaned successfully!")

# Example usage
clean_text_file("example.txt")
```

Q50. Write a Python program to count how many sentences are in a paragraph.

In [48]:
```python
def count_sentences_manual(paragraph):
    count = 0
    for char in paragraph:
        if char in ".!?":
            count += 1
    return count
print("Number of sentences:", count_sentences_manual("Hello world! How are you
```

```
Number of sentences: 3
```